NYC Taxi Medallion Data Pipeline

An Automated ELT Pipeline with Apache Airflow & PostgreSQL

Janidu Kodithuwakku (001) | Data Engineering CW1

1. Problem Statement and Dataset

1.1 Problem Statement

The New York City Taxi & Limousine Commission (TLC) publishes massive monthly trip-record datasets for Yellow and Green taxi fleets. These datasets arrive as raw, denormalised Apache Parquet files containing millions of records per month, with inconsistencies such as negative trip distances, null timestamps, and heterogeneous schemas between taxi types. The challenge is to design and implement a production-grade, automated Extract–Load–Transform (ELT) pipeline that:

1. Ingests raw data at scale into a relational database.
2. Cleans and validates records, removing duplicates and invalid entries.
3. Transforms data into a normalised, analytics-ready dimensional model.
4. Produces actionable business insights via analytical SQL queries.
5. Orchestrates the entire workflow with proper dependency management.

1.2 Dataset Description

The pipeline processes the official NYC TLC Trip Record Data[1]:

Table 1: Source Dataset Summary

File	Format	Description
`yellow_tripdata_YYYY-MM.parquet`	Parquet	Yellow taxi trip records (~65–70 MB each; ~2 M+ rows per month)
`green_tripdata_YYYY-MM.parquet`	Parquet	Green taxi trip records (~1 MB each; ~22 K+ rows per month)
`taxi_zone_lookup.csv`	CSV	265 taxi zone reference records (Borough, Zone, Service Zone)

2. System Architecture

The system follows a containerised microservices architecture orchestrated via Docker Compose. All services Airflow (Webserver, Scheduler, Worker, Triggerer), PostgreSQL, and Redis run as isolated containers with shared volumes for DAGs, data, and logs.

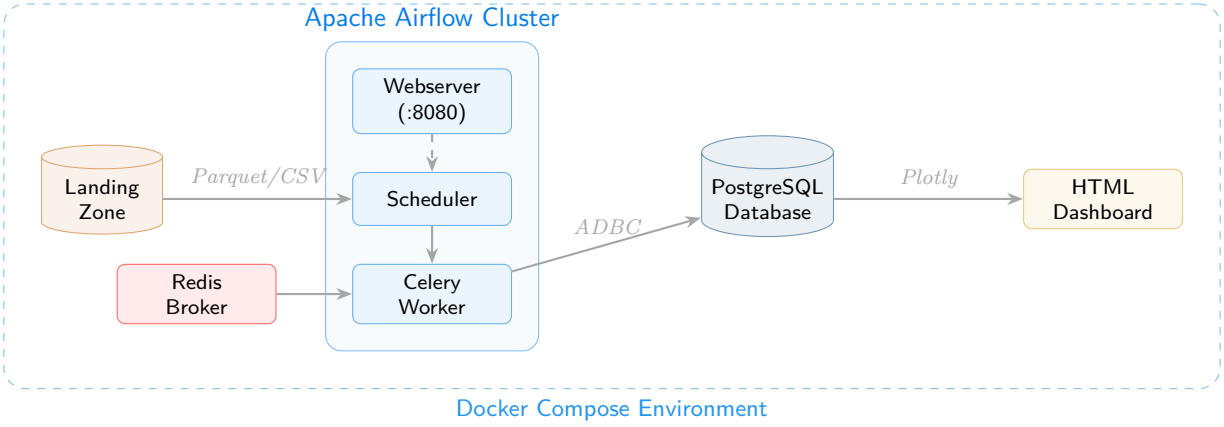

Figure 1: System Architecture: Containerised Deployment

[1] https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Key architectural decisions:

- CeleryExecutor with Redis broker enables horizontal scaling of task workers.
- Polars + ADBC (Arrow Database Connectivity) replaces the conventional Pandas + psycopg2 stack, providing up to 10× faster bulk ingestion through columnar Arrow-native transfers.
- Shared volumes mount `dags/`, `data/`, `logs/`, and `plugins/` for seamless hot-reloading of DAG code and data access across containers.

3. Database Schema

The database implements a Star Schema across three layers following the Medallion Architecture pattern:

- Bronze: Raw ingestion tables (`bronze.yellow_trips`, `bronze.green_trips`, `bronze.zone_lookup`). No constraints; schema mirrors source files.
- Silver: Normalised dimensional model with enforced Primary Keys, Foreign Keys, and typed columns.
- Gold: Pre-aggregated analytics tables with composite Primary Keys. `gold.hourly_spatial_demand`, `gold.revenue_summary_daily`

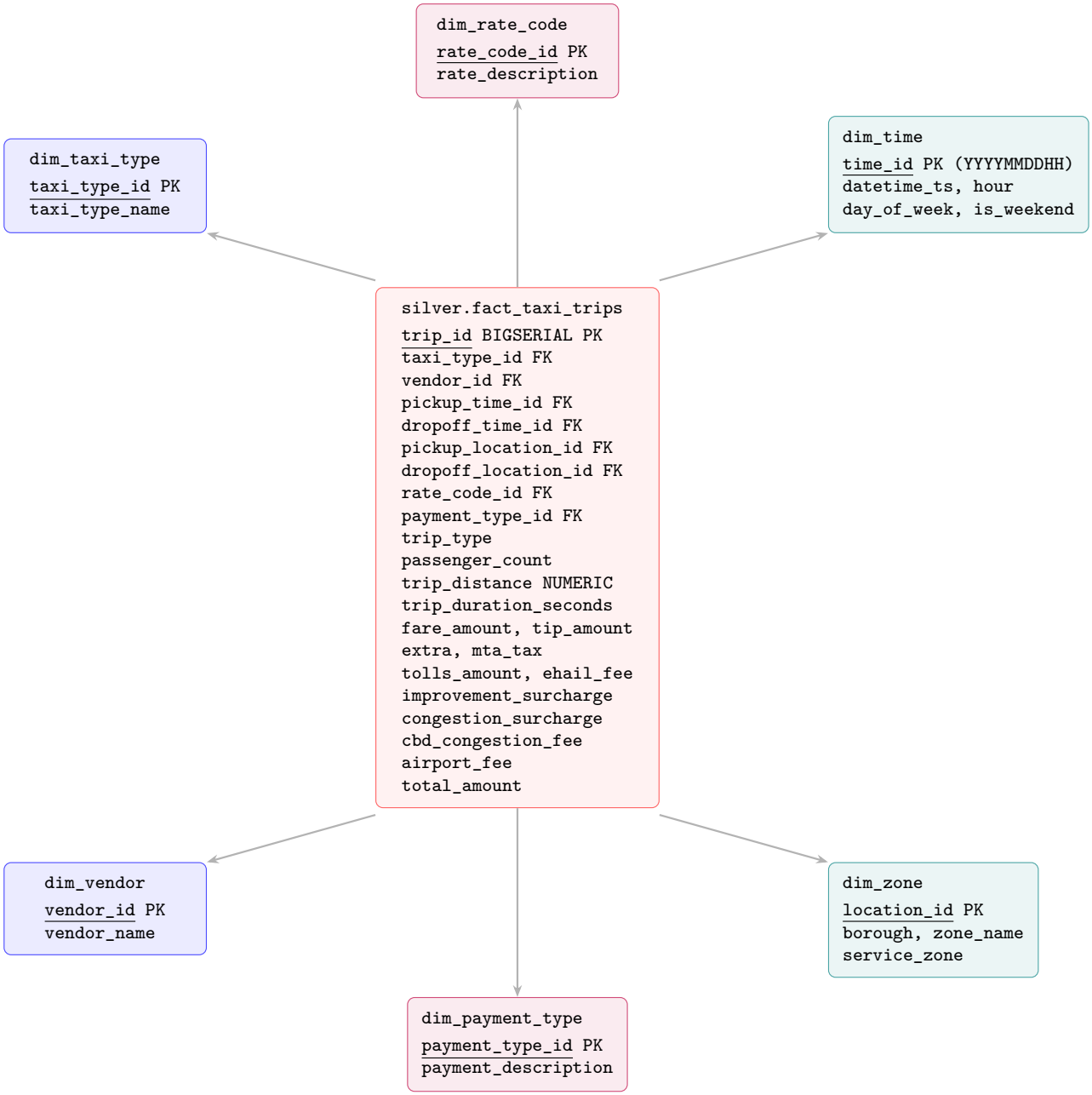

Figure 2: Star Schema ER Diagram: Silver Layer (6 Dimensions + 1 Fact Table)

The schema enforces referential integrity through Foreign Keys and optimises query performance with B-tree indexes on `pickup_time_id`, `pickup_location_id`, and `taxi_type_id`.

4. ETL Workflow

The pipeline follows an ELT (Extract–Load–Transform) pattern rather than traditional ETL, leveraging PostgreSQL's processing power for in-database transformations.

4.1 Extract & Load (Bronze Layer)

Raw Parquet files are detected via Airflow `FileSensor` operators, then ingested into PostgreSQL using Polars with the ADBC (Arrow Database Connectivity) engine. This zero-copy, columnar transfer protocol bypasses row-by-row insertion entirely, enabling bulk loading of 2 M+ records in seconds rather than minutes.

4.2 Clean & Validate

Data quality is enforced at two levels:

1. Post-load DQ checks: SQL assertions count records where `trip_distance < 0` or `total_amount < 0`, providing immediate feedback on data integrity.
2. Transformation-time filtering: The Silver insertion query applies `WHERE` clauses to exclude null timestamps (`IS NOT NULL`) and invalid distances (`trip_distance >= 0`). Duplicate time dimension entries are handled via `ON CONFLICT DO NOTHING`.

4.3 Transform (Silver Layer)

Three distinct transformations are applied during the Bronze-to-Silver migration:

1. Date Formatting & Feature Extraction: Timestamps are decomposed into a Time Dimension with derived fields: `time_id` (format `YYYYMMDDHH24`), `hour`, `day_of_week` (ISO: 1=Mon, 7=Sun), and a boolean `is_weekend` flag using `EXTRACT(ISODOW)`.
2. Derived Metric: Trip Duration: A new column `trip_duration_seconds` is computed dynamically: `EXTRACT(EPOCH FROM (dropoff - pickup))`.
3. Schema Unification: Yellow and Green taxi schemas are harmonised via `UNION ALL`, with NULL-casts for non-overlapping columns (`Airport_fee` for Green, `ehail_fee` and `trip_type` for Yellow), producing a single unified fact table.

4.4 Aggregate (Gold Layer)

Five analytical queries are executed against the Silver layer to generate business-intelligence insights. Results are rendered into an interactive HTML dashboard using Plotly.js charts generated by the `generate_report.py` module.

5. Airflow DAG Explanation

The DAG (`dynamic_nyc_taxi_medallion`) is a monthly-scheduled workflow containing 13 tasks organised into logical `TaskGroups` with carefully designed dependencies.

Key design features of the DAG:

- Parallel ingestion: Yellow, Green, and Zone data streams execute concurrently, maximising throughput.
- FileSensors: Dynamically wait for monthly Parquet files using Jinja-templated paths (`data_interval_start.strftime('%Y-%m')`), enabling fully automated monthly processing.
- Idempotency: The Silver fact table is truncated before each reload (`TRUNCATE TABLE`), while dimension tables use `ON CONFLICT` clauses, ensuring safe DAG reruns.
- Externalised SQL: All transformation and aggregation logic resides in dedicated `.sql` files under `dags/sql/`, keeping the DAG Python code clean and maintainable.
- Automatic retry: Tasks are configured with 2 retries and a 5-minute delay.

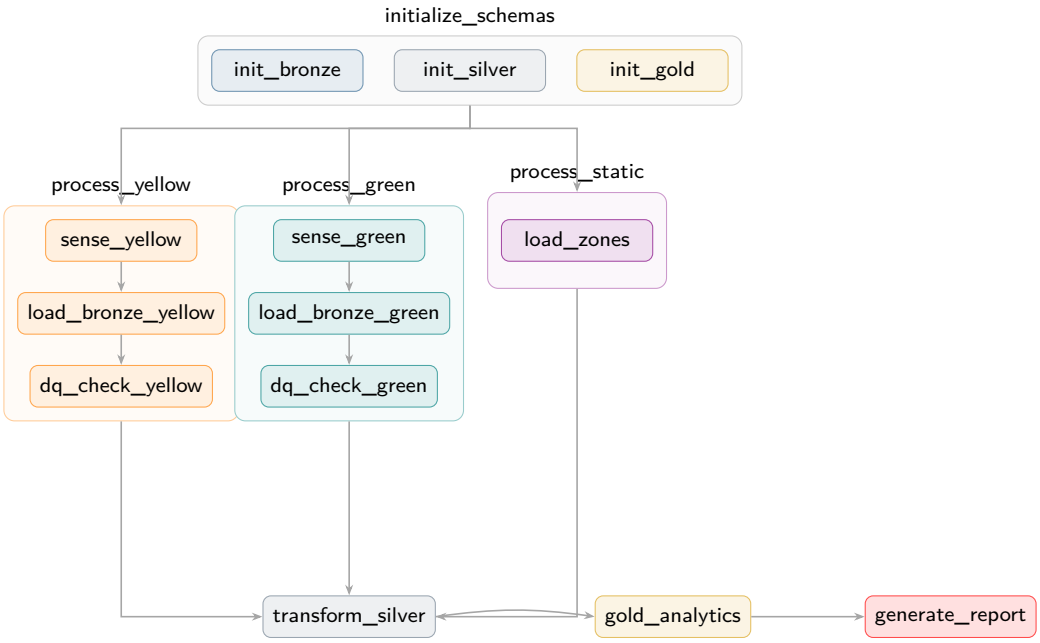

Figure 3: Airflow DAG Structure: Task Dependencies and Parallel Streams

6. Sample Query Results

The following tables present results from the pipeline's actual run on the May 2026 dataset (4,135,757 total trips).

6.1 Query 1: Fleet Rebalancing - Net Spatial Flow

Identifies zones "draining" vehicles (more pickups than dropoffs), critical for fleet repositioning logistics.

Table 2: Top 5 Vehicle-Deficit Zones

Borough	Zone	Departures	Arrivals	Net Flow
Queens	JFK Airport	149,329	37,246	−112,083
Queens	LaGuardia Airport	106,419	40,353	−66,066
Manhattan	Penn Stn / Madison Sq	113,930	76,404	−37,526
Manhattan	Midtown Center	161,323	138,621	−22,702
Manhattan	Upper East Side S.	198,023	178,497	−19,526

6.2 Query 2: CBD Congestion Fee Revenue

Evaluates the financial impact of NYC's Congestion Relief Zone surcharge.

Table 3: Congestion Fee Revenue by Borough (Top 5)

Borough	Type	Trips	Revenue ($)	Fees ($)	%
Manhattan	Yellow	2,547,071	70,496,895	1,910,303	2.71
Queens	Yellow	126,574	10,627,686	94,931	0.89
Brooklyn	Yellow	24,448	1,024,760	18,336	1.79
Manhattan	Green	2,286	98,860	1,715	1.73
Brooklyn	Green	984	47,112	738	1.57

6.3 Query 3: Traffic Velocity - Peak vs. Off-Peak

Calculates effective travel speed (MPH) to identify congestion bottlenecks. Key finding: average speed drops to 8.06 MPH at 4 PM on weekdays (severe gridlock) versus 18.0 MPH at 11 PM.

6.4 Query 4: Airport Corridor Market Share

Table 4: Airport Dropoff Market Share

Airport	Type	Dropoffs	Avg Fare ($)	Avg Dist (mi)
LaGuardia	Yellow	39,127	69.93	11.87
JFK	Yellow	36,900	85.20	12.90
Newark	Yellow	7,726	134.80	16.31
LaGuardia	Green	1,226	40.17	5.33
JFK	Green	346	75.18	215.04

6.5 Query 5: Tipping Elasticity by Trip Distance

Table 5: Tipping Behaviour by Distance Bracket (Credit Card Only)

Distance Bracket	Trips	Avg Tip ($)	Tip %
Micro (0–2 mi)	1,572,468	2.93	28.46%
Short (2–5 mi)	701,201	4.31	21.91%
Medium (5–10 mi)	253,522	6.76	18.11%
Long (10+ mi)	229,946	10.83	16.36%

Insight: While absolute tip amounts increase with distance, the tip-as-percentage-of-fare *decreases* monotonically; passengers tip a proportionally higher share on short trips.

7. Challenges and Lessons Learned

7.1 Challenges Encountered

1. Schema heterogeneity: Yellow and Green taxis have different column sets (`Airport_fee` exists only in Yellow; `ehail_fee` and `trip_type` only in Green). This was resolved via explicit `NULL::TYPE` casts during the `UNION ALL` unification in the Silver transformation.
2. ADBC driver compatibility: The `adbc-driver-postgresql` package required careful version pinning in the Docker environment via `_PIP_ADDITIONAL_REQUIREMENTS`. Initial attempts with mismatched Arrow/ADBC versions caused silent data truncation.
3. Memory management at scale: Processing 4 M+ records through Polars DataFrames required monitoring Docker container memory limits. Polars' lazy evaluation and streaming capabilities were essential to avoid out-of-memory errors.
4. Jinja templating in FileSensor: Airflow's double-curly-brace Jinja syntax conflicts with Python f-strings. This was resolved using quadruple braces (`{{{{ }}}}`) inside f-strings for proper template rendering.
5. Foreign Key constraint ordering: The Silver fact table insertion fails if dimension tables (especially `dim_time`) are not fully populated first. This was solved by carefully ordering the SQL statements within the transformation script; dimensions are populated before the fact table.

7.2 Lessons Learned

1. ELT over ETL: Pushing transformation logic into SQL (inside the database) is significantly more efficient than extracting data, transforming in Python, and reloading. PostgreSQL's query optimiser handles joins across millions of rows far better than application-level code.
2. Medallion Architecture pays off: The Bronze/Silver/Gold layering provides clear separation of concerns, simplifies debugging (raw data is always preserved), and enables independent schema evolution at each layer.
3. Polars + ADBC is production-ready: Replacing Pandas with Polars and psycopg2 with ADBC reduced ingestion time by an order of magnitude. The Arrow-native columnar transfer eliminates serialisation overhead entirely.

4. Externalise SQL from DAGs: Keeping SQL in separate `.sql` files improves readability and version control. It also lets database engineers modify transformations without touching Python.
5. Idempotency is non-negotiable: Every task in the DAG must be safely re-runnable. Using `ON CONFLICT DO NOTHING/UPDATE` for dimensions and `TRUNCATE` before fact-table loads guarantees this property.

Technology Stack Summary

Component	Technology
Orchestration	Apache Airflow 2.9.0 (CeleryExecutor)
Database	PostgreSQL 13
Data Processing	Polars (DataFrame), ADBC (database connector)
Visualisation	Plotly.js (interactive HTML dashboard)
Containerisation	Docker & Docker Compose
Message Broker	Redis
Language	Python 3, SQL

* * *